

AI Identity Security

Capstone Project

Project 2: Build and Secure the Auth0-Backed AI Application

Submitted By

Sri Vidya Praveena

Project Description

SecureNova Inc. has deployed an AI-powered customer service platform in which an LLM agent, backed by Auth0 for identity management, calls internal REST APIs, retrieves knowledge from a RAG pipeline, and invokes MCP server tools on behalf of authenticated users. This project focuses on building and securing the Auth0 identity layer that underpins that platform. It covers registering and configuring the human-facing and machine-facing applications in Auth0, implementing the OAuth 2.0 Authorization Code flow with PKCE for user login, defining scope-based access control between a standard user role and an admin role, enforcing multi-factor authentication and brute-force/suspicious-IP protection, issuing and rotating short-lived machine-to-machine credentials, and using an Auth0 Action to bind a custom agent identity claim into every issued token. Together, these controls demonstrate how Zero Trust and least-privilege principles from traditional IAM can be applied to an AI system's non-human and human identities alike.

Objectives

1. Set up a free Auth0 developer tenant and register two applications: a Regular Web Application for the AI chat service (human/user login) and a Machine-to-Machine application for the AI agent's backend-to-API calls.
2. Create a custom API in Auth0 and define two scopes `read:ai-data` (user-level) and `write:admin` (admin-level) to enforce least-privilege, role-based access on every issued access token.
3. Implement the OAuth 2.0 Authorization Code flow with PKCE for the Regular Web Application, and verify at jwt.io that issued tokens are signed with RS256 and carry the correct `aud`, `iss`, `exp`, and `scope` claims.
4. Demonstrate that a token holding only the `read:ai-data` scope is rejected with an HTTP 403 when it is used to call the admin-scoped (`write:admin`) endpoint, proving scope enforcement works as intended.
5. Enable TOTP-based Multi-Factor Authentication for all users and capture the enrollment/challenge flow, adding a second authentication factor beyond password.
6. Configure Auth0 Attack Protection Brute-Force Protection and Suspicious IP Throttling - to defend user accounts against credential-stuffing and repeated password-guessing attacks.
7. Write and deploy an Auth0 post-login Action that injects a custom `agent_id` claim into the ID token and access token, so every authenticated session can be traced back to a specific agent identity.
8. Write a Python script that obtains a short-lived (60-second TTL) Machine-to-Machine access token, uses it successfully against a protected `/chat` endpoint, and then confirms that Auth0 correctly rejects the same token with a 401 Unauthorized once it has expired validating short-lived credential rotation.
9. Apply custom branding to the Auth0 Universal Login page so the authentication experience is recognizable as SecureNova's own, rather than the unbranded Auth0 default.

Auth0 Application Page (Screenshot-1)

This is the Applications list in the Auth0 Dashboard, confirming both required applications were successfully registered in the tenant. “AI Agent M2M” is registered as a Machine to Machine application, used by the AI agent to obtain access tokens programmatically via the Client Credentials grant for backend-to-API calls. “AI Chat Service” is registered as a Regular Web Application, used for interactive user login through the Authorization Code + PKCE flow. The tenant's default application and the securenova-ai-api test application (auto-created by Auth0 when the API was defined) are also visible, but the two applications relevant to this task are the ones named above.

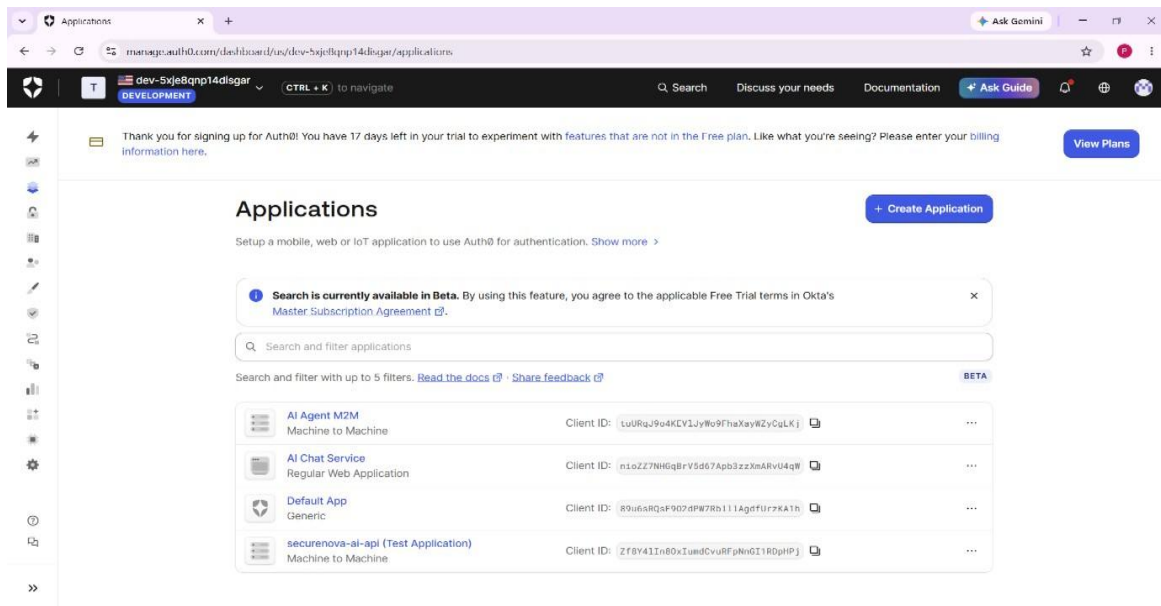


Fig1:Auth0 dashboard with two applications

Auth0 APIs Page -Custom Scopes (Screenshot2):

This is the Permissions tab of the custom API registered as `securenova-ai-api` in the Auth0 Dashboard. Two scopes were defined here to support role-based access control on the `/chat` and `/admin` endpoints: `read:ai-data`, granted at the user level, and `write:admin`, reserved for admin-level access. Defining these scopes on the API is what allows Auth0 to issue access tokens whose scope claim reflects exactly what the calling client or user is permitted to do, which is verified against a protected endpoint in Screenshot 3.

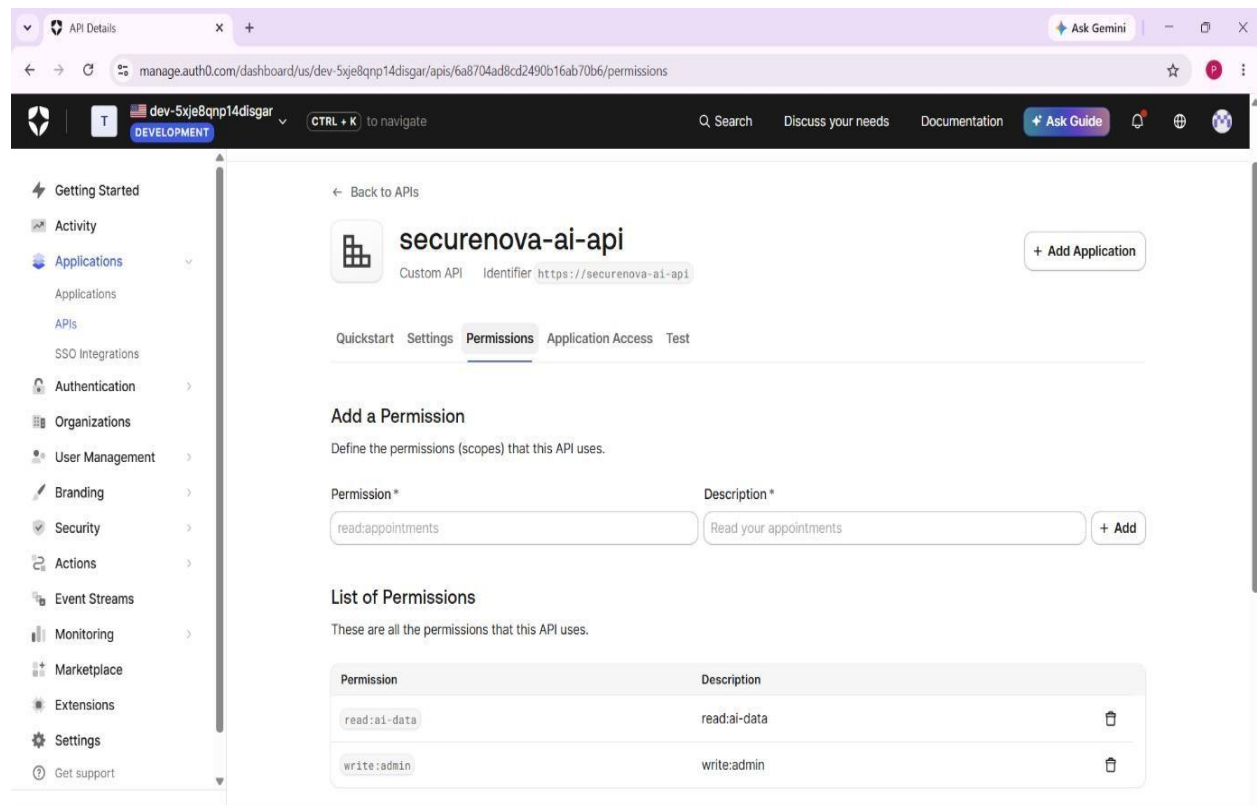


Fig2:Auth0 Dashboard with given scopes in securenova-ai-api

Decoded JWT at jwt.io - RS256, Claims & Custom agent_id (Screenshot-3)

These two captures together verify the issued token end-to-end. The first shows an access token pasted into jwt.io: the decoded header confirms it is signed with the RS256 algorithm, and the decoded payload shows the required iss, sub, aud, iat, exp, and scope claims, along with the custom agent-id claim added by the deployed Action. The second capture is from a small local PKCE demo application (localhost:3000) used to drive the Authorization Code + PKCE login flow and display the resulting tokens: it shows the same agent-id claim present in both the access token and the ID token, along with standard ID token claims (nickname, name, picture) returned for the logged-in user, and a link used to call the admin-scoped endpoint to test scope enforcement (its result is discussed under Screenshot 9).

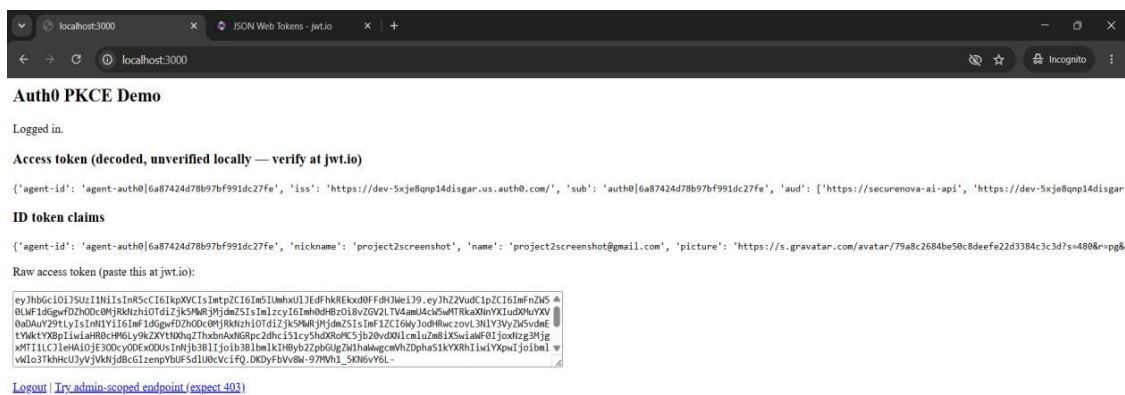
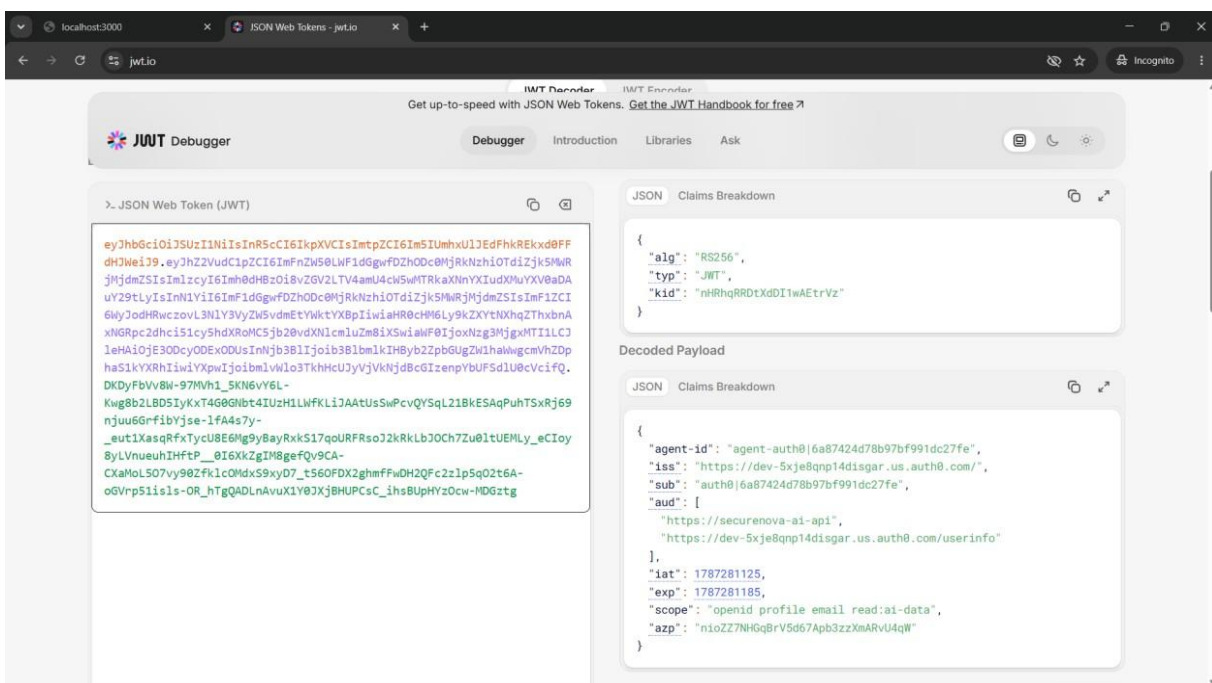


Fig 3: Decoded JWT at jwt.io showing RS256 header and claims

Auth0 Universal Login Page - Custom Branding (Screenshot-4)

These captures show the Auth0 Universal Login page rendered in the browser during an actual login attempt, with custom branding applied through Auth0 Dashboard → Branding → Universal Login. The page displays the custom application name “SecureNova Inc” in place of the default Auth0 header, along with a customized green “Continue” button, on a dark themed background. The second capture shows the same branded page with a test account's email and password entered, immediately before submitting the login confirming the branding is applied consistently through the full login interaction, not only on the initial page load.

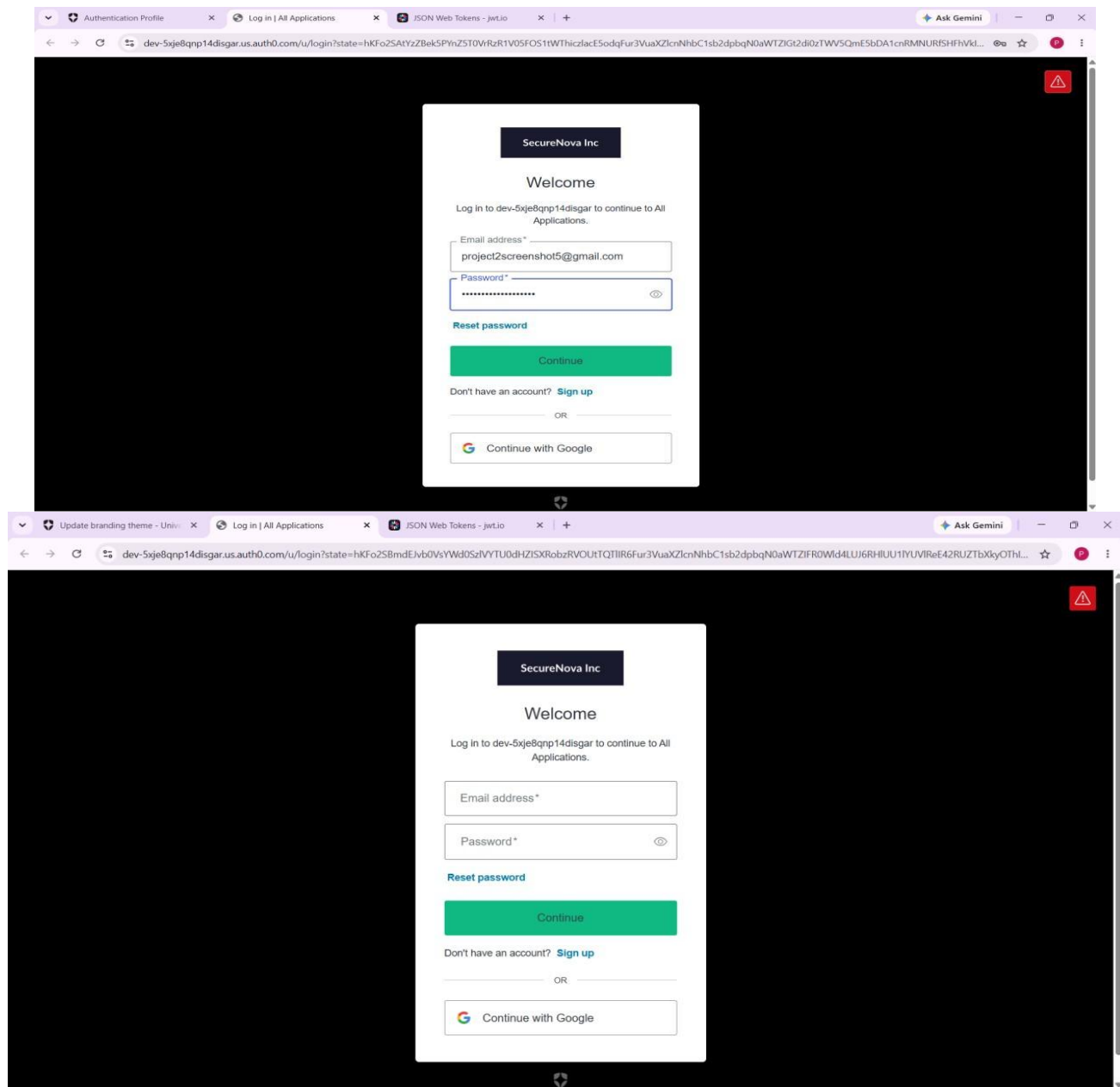


Fig 4: Auth0 Universal Login page with custom branding applied

MFA Enrollment - TOTP QR Code (Screenshot-5)

This screenshot was captured after enabling Multi-Factor Authentication via Auth0 Dashboard → Security → Multi-factor Auth, with One-time Password (TOTP) turned on and the policy set to always require a second factor. It shows the enrollment screen presented on first login for a test account: a QR code that the user scans with an authenticator app, followed by a field to enter the resulting one-time code before access is granted. This confirms MFA is actively enforced at login, not just enabled in configuration.

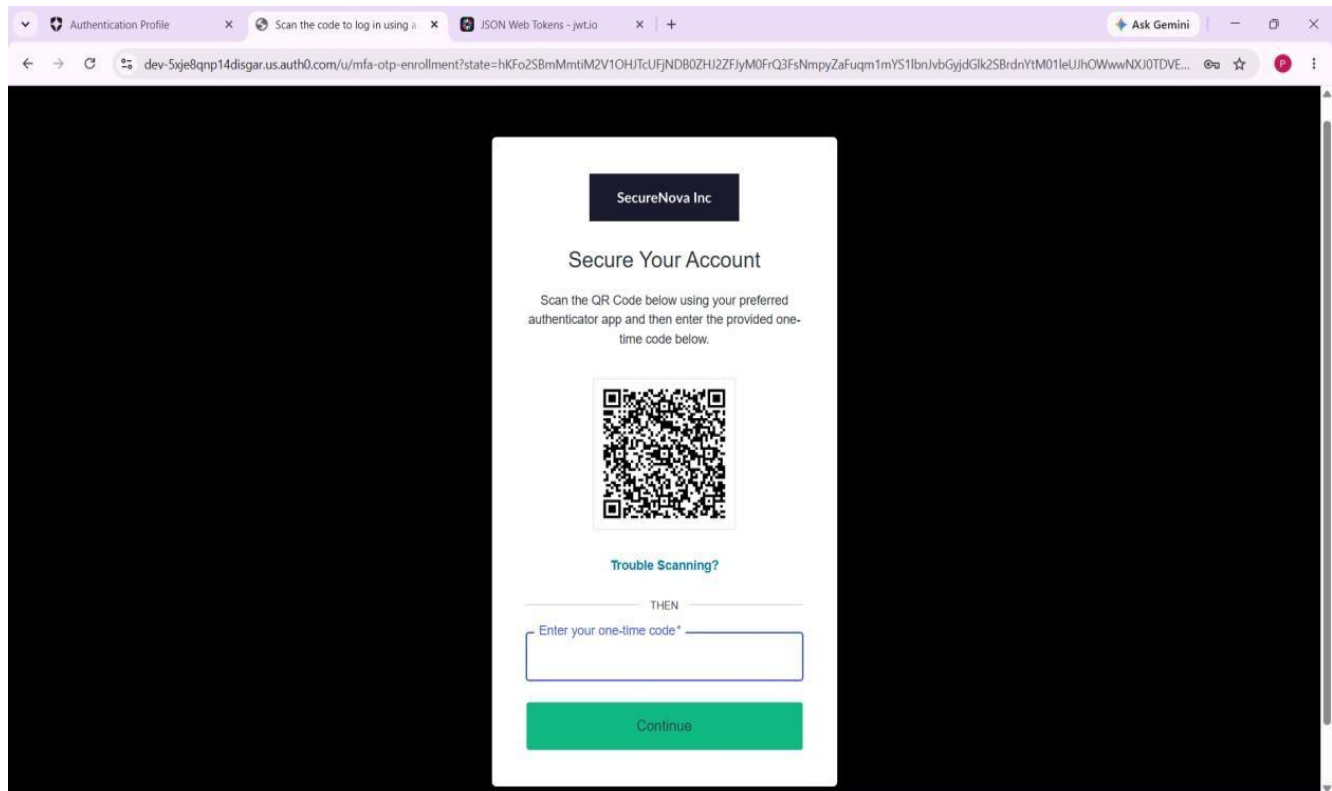


Fig 5: MFA TOTP enrollment screen with QR code

Auth0 Actions Editor-Post-Login Action(Custom agent_id Claim)(Screenshot 6)

This screenshot shows the Actions editor for the custom “Add Agent ID Claim” Action, with its trigger set to Login / Post Login. The deployed code runs after every successful login: it builds an agentId value from the user's app_metadata (falling back to a value derived from their user_id), and then calls `api.idToken.setCustomClaim()` and `api.accessToken.setCustomClaim()` to attach that value under a namespaced agent-id claim on both the ID token and the access token. The “Action is up to date” status in the top-right confirms this version is deployed, and it has also been attached to the tenant's Login flow so it executes on every user authentication its output is what appears as the agent-id claim in the decoded tokens shown in Screenshot 3.

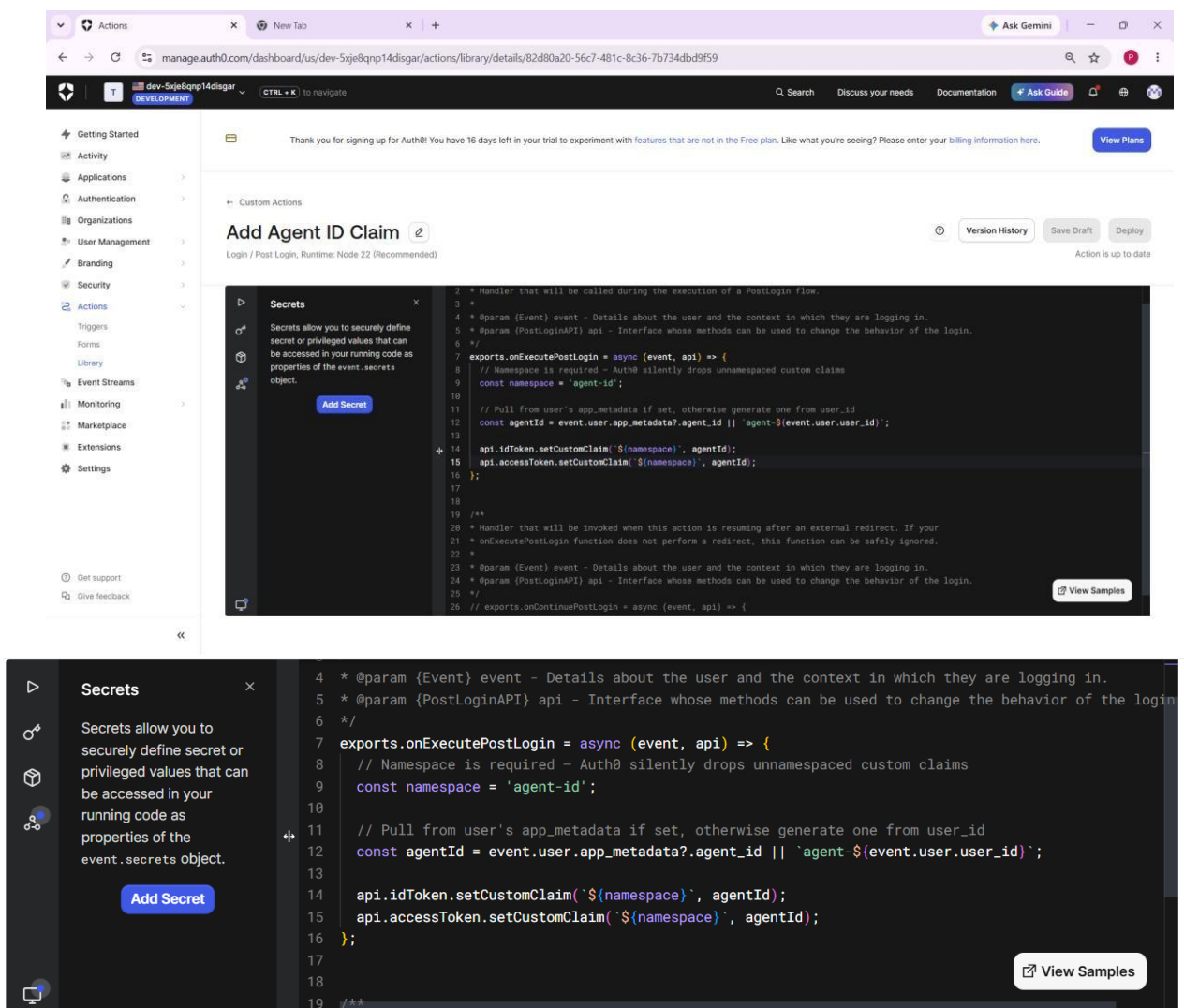


Fig 6: Auth0 Actions editor showing the Add Agent ID Claim post-login Action

Auth0 Attack Protection: Brute-Force & Suspicious IP Throttling (Screenshot 7):

This screenshot shows the Attack Protection overview in Auth0 Dashboard → Security → Attack Protection, confirming that both Suspicious IP Throttling and Brute-force Protection are Enabled for the tenant. Brute-force Protection was configured with a maximum of 10 failed login attempts before Auth0 blocks further attempts against that account, with user notifications turned on; Auth0 enforces the block-duration window itself and does not expose it as a separately configurable field on the dashboard. Together these two controls protect user accounts against credential-stuffing and repeated password-guessing attacks by automatically throttling or blocking further attempts once the threshold is exceeded.

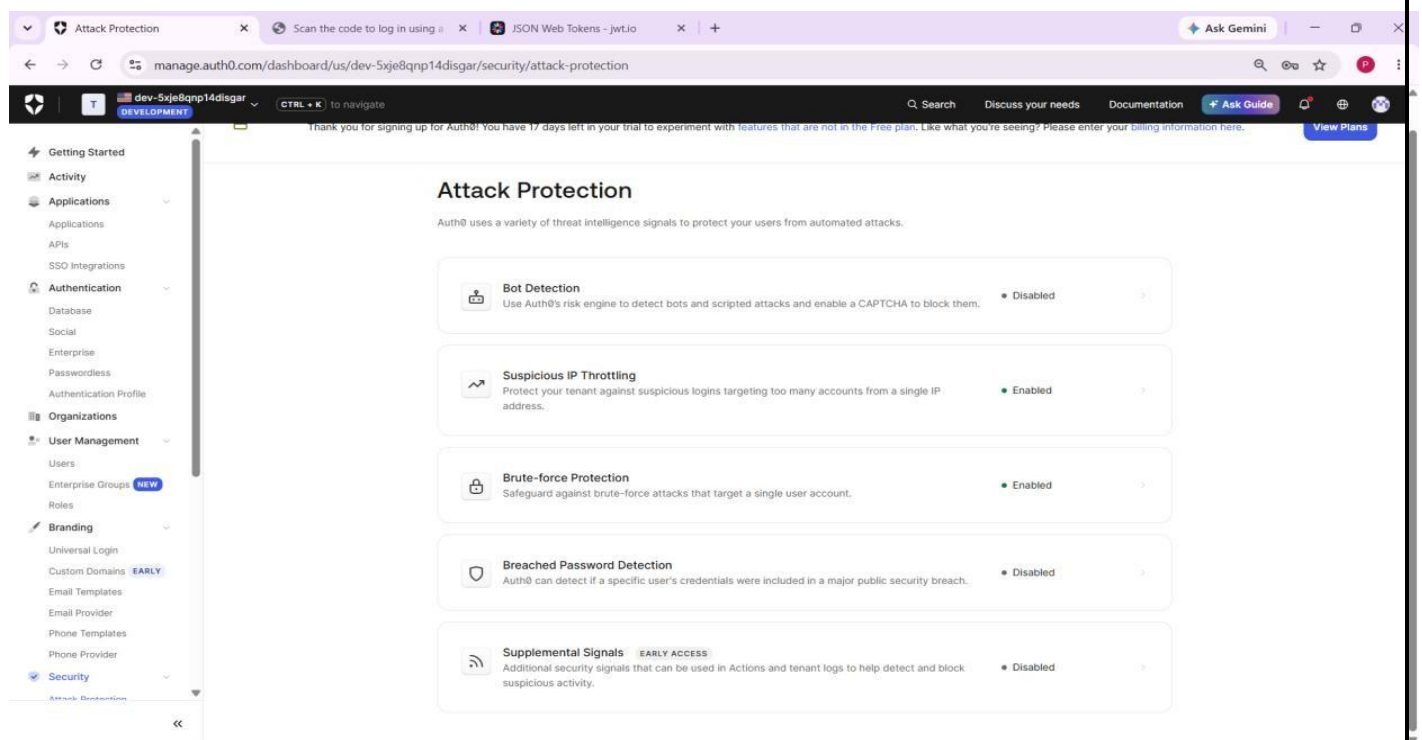


Fig 7: Attack Protection panel with Brute-Force Protection and Suspicious IP Throttling enabled

Expired-Token Enforcement - 401 Unauthorized on Replay (Screenshot 8) :

This capture shows the result of calling the admin-protected endpoint (localhost:3000/call-admin) using a previously issued access token after its 60-second expiry window had passed. The response returned is a 401 Unauthorized with the body {"error": "Token expired"}, confirming that Auth0's issued token is time-bound and that the resource server correctly rejects it once expired rather than trusting a cached credential indefinitely. This result was produced through the local test harness's browser-based call rather than a terminal session; the equivalent standalone Python script (credential_rotation_test.py), which performs the same fresh-token-then-expired-token replay against a /chat endpoint from the command line with timestamped output, is included separately as supporting evidence of the same TTL-enforcement behavior.

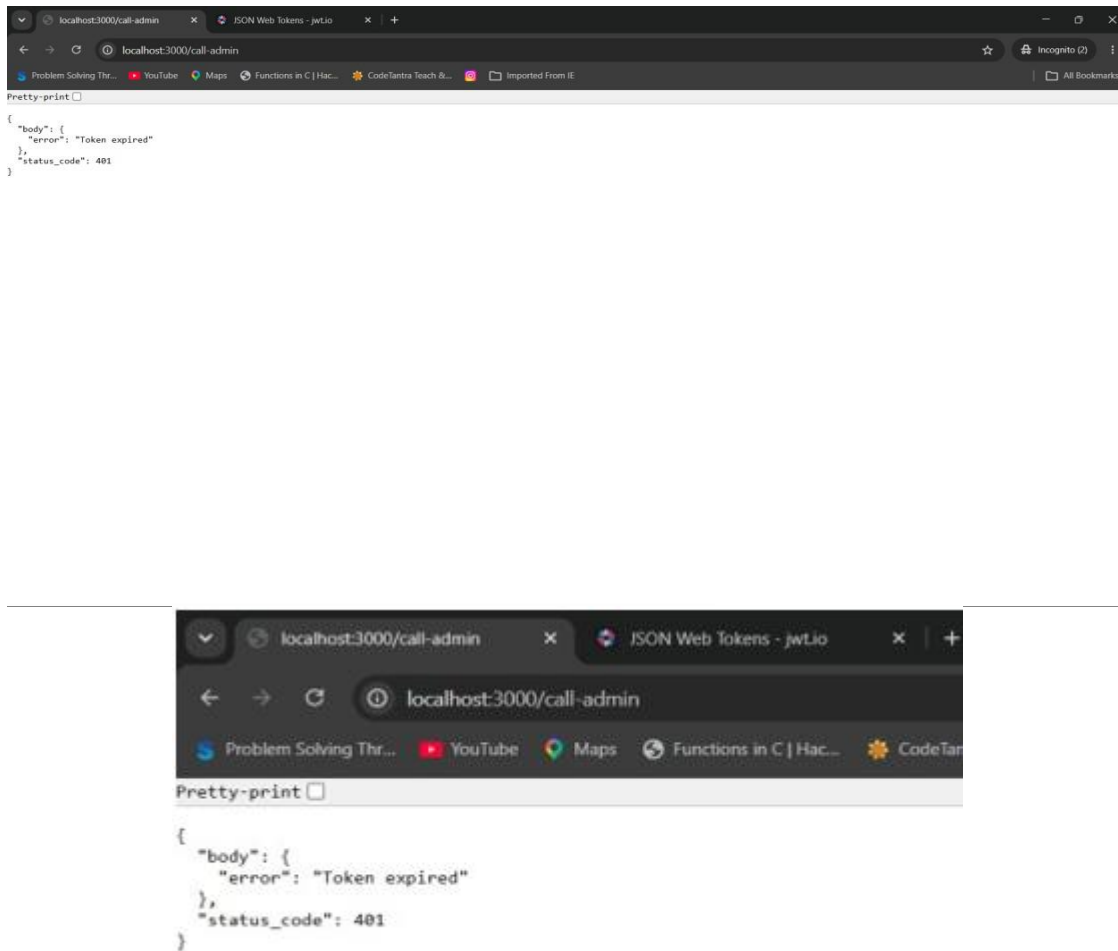


Fig 8: 401 Unauthorized response returned on replay of an expired access token